

作业 HW2* 实验报告

姓名：李盛鹏 学号：2351136 日期：2024 年 10 月 18 日

- # 实验报告格式要求按照模板（使用 Markdown 等也请保证报告内包含模板中的要素）
- # 对字体大小、缩进、颜色等不做强制要求（但尽量代码部分和文字内容有一定区分，可参考 vscode 配色）
- # 实验报告要求在文字简洁的同时将内容表示清楚
- # 报告内不要大段贴代码，尽量控制在 20 页以内

1. 涉及数据结构和相关背景

题目或实验涉及数据结构的相关背景

- 1.线性表：可以使用数组或链表来存储多项式的系数，便于访问和修改。
- 2.数组或列表：用于存储项的值，便于逐项累加。
- 3.栈：适合实现“后进先出”的牌堆操作。
- 4.队列：适合实现“先进先出”的操作。（当然栈和队列虽然涉及，但是不能使用）
- 5.时间复杂度：加法和乘法操作通常为 $O(n)$ ，其中 n 为多项式的最高次幂。

2. 实验内容

2.1 最大子段和（题目名字）

1.列车进站

2.1.1 问题描述

简短地描述题目（可复制题目要求）

每一时刻，列车可以选择从入口进车站、直接从入口到出口，或从车站到出口。给定一组进站序列和多组出站序列，判断是否能通过这些路径将所有列车顺利送出。

2.1.2 基本要求

简短地描述题目要求

题目要求判断列车在进出站过程中是否能按照给定的进站序列顺利出站。每一时刻，列车可以选择从入口进车站、直接进入出口，或从车站进入出口。对于每个出栈序列，输出"yes"表示可以顺利出站，输出"no"表示不可以。

2.1.3 数据结构设计

图、描述均可

贴出数据结构定义代码（变量、结构体或类）并使用注释辅助说明

例如

```
// Definition for a binary tree node.
```

```

struct TreeNode {
    int data;           // 节点元素
    TreeNode *lchild;   // 左孩子节点指针
    TreeNode *rchild;   // 右孩子节点指针
    TreeNode(int x = 0, TreeNode *left = nullptr, TreeNode *right = nullptr);
    ~TreeNode();
};

```

```

/* 顶指针 */
int top = -1;
char station[max_element]; // 车站栈
int entry_index = 0;        // 进站序列索引
int exit_index = 0;         // 出站序列索引

/* 定义进站序列和出栈序列 */
char entry[max_element];    // 进站序列
char exit[max_element];     // 出栈序列

/* 函数：检查是否能够实现出栈序列 */
function canExit(entry, exit) :
    while exit_index < length(exit) :
        // 如果车站不为空且出站车与栈

```

2.1.4 功能说明（函数、类）

推荐使用代码+注释的形式，清晰描述函数功能、输入内容和输出/返回内容

不要直接贴函数实现代码，请选择关键代码/伪代码展示并用注释辅助说明

例如

```

/**
 * @brief      二叉树指令构造
 * @param      order    构造指令列表
 * @param      index    当前节点序号
 * @return     子树根节点指针
 */
TreeNode *buildTreeDFS(const std::vector<int> &order, int &index) {
    if (子树为空) {
        返回空指针 // 返回空指针给父节点
    }
    // 构造以当前节点为根节点的子树并返回
    return new TreeNode(order[index ++], buildTreeDFS(order, index),
        buildTreeDFS(order, index));
}

```

```

/*
    如果能从这个循环中走出,则说明符合题意
*/
while (exit_index < exit_num)
{
    /*-----
    入栈逻辑
    1. 入栈顺序有几个元素就最多只能进多少元素
    2. 只有栈顶元素不等于当前出栈元素时才可以入栈
    -----*/
    while (entry_index < entry_num && station[top] != exit[exit_index])
    {
        // 入栈操作
        station[++top] = entry[entry_index]; // 将元素入栈
        entry_index++; // 移动到下一个入栈元素
    }

    /*-----
    由于前面的while有两个条件, 要么入栈序列还有数,
    要么栈顶等于出栈序列
    如果出栈条件不满足, 说明入栈序列已经空了, 则返回false
    -----*/
}

```

```

/* 出栈逻辑 */
if (top != -1 && station[top] == exit[exit_index])
{
    // 满足出栈条件, 执行出栈
    top--; // 栈顶指针下移
    exit_index++; // 移动到下一个出栈元素
}
else
{
    // 出栈条件不满足, 返回false
    return false;
}

/* 如果循环正常结束, 说明可以按照指定的出栈顺序出栈 */
return true;

```

2.1.5 调试分析（遇到的问题解决方法）

- # 简要描述调试过程
- # 可以使用图片辅助说明

遇到的问题：

出栈逻辑错误：在出栈判断时，可能会因为栈顶元素与当前出栈元素不匹配而提前返回 false，导致无法正确判断出栈序列的有效性。

输入读取问题：在处理输入时，程序未能处理输入结束标志（EOF），可能导致无限循环或未能正确读取最后一行数据。

字符数组长度限制：使用固定长度的数组来存储入栈和出栈序列可能导致溢出，尤其是在输入较长序列时，未做长度检查。

解决方法：

修改出栈条件：在出栈操作中，如果栈不为空且栈顶元素与当前出栈元素不匹配，则应尝试继续入栈，直到找到匹配的出栈元素或入栈序列耗尽。

改进输入处理：使用 `std::getline()` 读取输入，并检查是否达到了文件结束标志，以确保所有出栈序列都被正确读取。

2.1.6 总结和体会

可以说说做此题的收获、分析这道题目的难点和易错点（数据边界、对算法效率的要求等）

收获：在解决这个列车进站问题时，我收获了对栈结构应用的深入理解，以及如何将实际场景转化为程序逻辑。该题目的难点在于正确处理入栈和出栈的顺序，以及确保栈操作不超出边界，这需要仔细分析每个条件的满足情况。易错点主要集中在出栈逻辑上，特别是当栈顶元素与出栈目标不匹配时，容易忽略继续入栈的必要性，导致错误判断。

。

3.2 题目二

最长自串

3.2.1 题目描述

给定一个仅包含字符 '(' 和 ')' 的字符串，要求计算出最长的有效括号子串的长度及其起始位置。有效括号子串是指括号配对正确且连续的部分，如果存在多个最长子串，取第一个的起始位置。空串的起始位置规定为 0，字符串长度范围为 0 到 100,000。

3.2.2 题目要求

题目要求计算一个仅包含字符 '(' 和 ')' 的字符串中，最长有效括号子串的长度及其起始位置。如果存在多个最长子串，应返回第一个的起始位置。输入为一行字符串，输出应为子串的长度和起始位置，空串的起始位置固定为 0。

3.2.3 数据结构设计

```

// 定义 SElemType 为整数类型, 用于记录左括号的下标
typedef int SElemType;

// 定义顺序栈结构体 SqStack
struct SqStack
{
private:
    SElemType* base;           // 指向栈底的指针
    SElemType* top;            // 指向栈顶的指针
    int stacksize;             // 栈的容量

public:
    // 构造函数: 初始化空栈
    SqStack();

    // 析构函数: 销毁已有的栈
    ~SqStack();

    // 方法: 清空栈, 置为空栈
    Status ClearStack();

    // 方法: 获取栈顶元素

```

```

// 构造函数: 初始化空栈
SqStack();

// 析构函数: 销毁已有的栈
~SqStack();

// 方法: 清空栈, 置为空栈
Status ClearStack();

// 方法: 获取栈顶元素
Status Top(SElemType& e);

// 方法: 弹出栈顶元素
Status Pop(SElemType& e);

// 方法: 新元素入栈
Status Push(SElemType e);

// 方法: 判断栈是否为空
bool StackEmpty();
};

```

这个 SqStack 结构体定义了一个顺序栈, 使用整型指针来管理栈底和栈顶的位置。构造函数 SqStack() 初始化一个空栈, 设置栈底和栈顶指针。析构函数 ~SqStack() 用于释放栈占用的内存。ClearStack() 方法清空栈, 将其重置为初始状态, 而 Top() 方法可以获取当前栈顶的元素。Pop() 方法用于弹出栈顶元素, Push() 方法则用于将新元素推入栈中, 最后 StackEmpty() 方法用于检查栈是否为空。

3.2.4 功能实现

```

// 定义常量
const int STACK_INIT_SIZE = 100; // 初始栈大小
const int STACKINCREMENT = 10;   // 栈增长量
const int SOVERFLOW = -1;         // 内存溢出标志
const int OK = 0;                 // 操作成功标志
const int ERROR = -1;             // 操作失败标志

// 构造函数: 初始化空栈
Function SqStack()
    base = new(SElemType[STACK_INIT_SIZE]) // 分配初始内存
    If (base is null) Then
        exit(SOVERFLOW) // 内存分配失败, 退出
    top = base // 设置栈顶指针
    stacksize = STACK_INIT_SIZE // 设置栈容量

// 析构函数: 销毁已有的栈
Function ~SqStack()
    If (base is not null) Then
        delete base // 释放栈内存
    stacksize = 0 // 重置栈容量

// 清空栈, 置为空栈

```



```

// 清空栈, 置为空栈
Function ClearStack()
    If (base is not null) Then
        delete base // 销毁原有空间
    base = new(SElemType[STACK_INIT_SIZE]) // 重新分配内存
    If (base is null) Then
        exit(SOVERFLOW) // 内存分配失败, 退出
    top = base // 重置栈顶指针
    stacksize = STACK_INIT_SIZE // 重置栈容量
    return OK

// 获取栈顶元素
Function Top(out e)
    If (top == base) Then
        return ERROR // 栈为空, 返回错误
    e = *(top - 1) // 获取栈顶元素
    return OK

// 弹出栈顶元素
Function Pop(out e)
    If (top == base) Then
        return ERROR // 栈为空, 返回错误

// 弹出栈顶元素
Function Pop(out e)
    If (top == base) Then
        return ERROR // 栈为空, 返回错误
    e = *(--top) // 弹出栈顶元素
    return OK

// 新元素入栈
Function Push(e)
    If (top - base >= stacksize) Then // 检查栈是否满
        newbase = new(SElemType[stacksize + STACKINCREMENT]) // 扩大栈空间
        If (newbase is null) Then
            exit(SOVERFLOW) // 内存分配失败, 退出
        memcpy(newbase, base, sizeof(SElemType) * stacksize) // 复制原有数据
        delete base // 释放原有内存
        base = newbase // 更新栈底指针
        top = base + stacksize // 更新栈顶指针
        stacksize += STACKINCREMENT // 增加栈容量
    *top = e // 将新元素入栈
    top++ // 更新栈顶指针
    return OK

// 判断栈是否为空
Function StackEmpty()
    return (base == top) // 判断栈底指针是否等于栈顶指针

```

这段代码实现了一个顺序栈 (SqStack) 的基本操作, 以下是对其功能的五句话解释:

1. ****常量定义****: 代码首先定义了一些常量, 包括初始栈大小、栈增长量、内存溢出标志、操作成功和失败的标志, 这些常量用于后续的栈操作中, 以确保代码的可读性和可维护性。
2. ****构造函数****: 在构造函数中, 栈被初始化为一个指定大小的数组, 并设置栈顶指针和栈容量, 如果内存分配失败, 则程序会退出并返回内存溢出标志。
3. ****析构函数****: 析构函数负责释放栈占用的内存空间, 确保在栈不再使用时能够正确地清理资源, 并将栈容量重置为 0, 以避免内存泄漏。
4. ****栈操作****: 提供了多个栈操作函数, 包括清空栈、获取栈顶元素、弹出栈顶元素和入栈操作, 确保用户可以方便地对栈进行各种操作, 并在栈为空时返回错误。
5. ****栈状态检查****: 通过`StackEmpty`函数, 可以判断栈是否为空, 这对于在进行弹出或获取栈顶元素等操作之前进行有效性检查是非常重要的。

3.2.4 遇到的困难和解决方法

栈的使用: 在处理括号匹配时, 栈是一个非常有效的数据结构。初始时, 可能会对如何使用栈来存储左括号的索引感到困惑。解决方法是明确栈的操作: 当遇到左括号时, 将其索引压入栈中; 当遇到右括号时, 检查栈是否为空, 如果不为空, 则弹出栈顶元素并标记这对括号为有效。

有效括号的标记: 在遍历字符串时, 需要标记哪些括号是有效的。最初可能会忽略如何有效地记录这些有效括号的索引。解决方法是使用一个布尔数组 `valid`, 在每次成功匹配括号时, 将对应的索引标记为 `true`。

计算最长子串的逻辑: 在找到有效括号后, 如何计算最长的有效子串长度和起始位置可能会让人困惑。解决方法是遍历 `valid` 数组, 使用两个变量 `CurLen` 和 `MaxLen` 来跟踪当前有效子串的长度和最大有效子串的长度, 并在遇到无效括号时更新最大长度和起始位置。

处理边界情况: 需要考虑一些特殊情况, 例如空字符串或只有右括号的字符串。解决方法是在代码中添加条件判断, 确保在这些情况下能够正确返回长度为 0 和起始位置为 0 的结果。

内存管理: 在使用动态内存分配时, 可能会遇到内存泄漏或分配失败的问题。解决方法是使用 `nothrow` 来处理内存分配, 并在析构函数中确保释放所有分配的内存, 以避免内存泄漏。

3.2.5 心得体会

在解决“最长正确括号子串”问题的过程中, 我深刻体会到以下几点:

数据结构的重要性: 栈作为一种后进先出 (LIFO) 的数据结构, 在处理括号匹配问题时非常有效, 能够帮助我们轻松管理左括号的索引, 从而快速找到匹配的右括号。

边界条件的处理: 在编写代码时, 考虑到各种边界情况 (如空字符串、只有右括号的字符串等) 是至关重要的, 这样可以避免潜在的错误和异常情况, 确保程序的健壮性。

动态内存管理: 使用动态内存分配时, 必须小心处理内存的分配和释放, 以防止内存泄漏。通过合理的内存管理, 可以提高程序的性能和稳定性。

有效性标记的使用: 通过使用布尔数组来标记有效的括号, 可以简化后续的长度计算逻辑, 使得代码更加清晰易懂, 减少了复杂的条件判断。

调试与测试的重要性: 在实现算法后, 进行充分的测试和调试是必不可少的, 尤其是在

处理复杂的字符串时，确保代码在各种输入下都能正确运行，能够帮助我们发现潜在的问题并及时修复。

4.2 题目三

布尔表达式

4.2.1 题目描述

计算如下布尔表达式 $(V|V) \& F \& (F|V)$ 其中 V 表示 True，F 表示 False，| 表示 or，& 表示 and，! 表示 not（运算符优先级 not > and > or）

4.2.2 题目要求

布尔表达式计算：需要计算给定的布尔表达式，这些表达式由布尔值（V 表示 True，F 表示 False）和逻辑运算符（&表示与，|表示或，!表示非）组成，运算符的优先级为 not > and > or。

输入格式：输入来自文件，包含若干行（最多 20 行），每行是一个布尔表达式。表达式中的符号可以用任意空格分隔，也可以没有空格，且每个表达式的长度（符号个数）不超过 100 个字符。

数据范围：对于不同的数据集，表达式的复杂度逐步增加。最简单的情况只包含 V、F、&、|，而最复杂的情况还包括逻辑非运算符!和括号()。

处理空格：在解析表达式时，需要能够忽略符号之间的空格，以确保正确计算表达式的值。

随机测试数据：可以通过运行提供的 96.py 脚本生成随机测试数据，以便进行验证和测试。

4.2.3 数据结构设计

```
// 定义元素类型
typedef char SElemType; // 元素类型为字符

// 定义状态类型
typedef int Status; // 返回值类型，状态

// 定义状态常量
#define OVERFLOW -2 // 内存溢出
#define ERROR -1 // 错误状态
#define OK 0 // 操作成功

// 顺序栈结构体
struct sqstack
{
private:
    SElemType* top; // 栈顶指针
    SElemType* base; // 栈底指针
    int size; // 栈的大小
public:
    sqstack(); // 构造函数
    ~sqstack(); // 析构函数
    Status getTop(SElemType& e); // 获取栈顶元素
    Status pop(SElemType& e); // 删除栈顶元素
```

```
// 顺序栈结构体
struct sqstack
{
private:
    SElemType* top; // 栈顶指针
    SElemType* base; // 栈底指针
    int size; // 栈的大小
public:
    sqstack(); // 构造函数
    ~sqstack(); // 析构函数
    Status getTop(SElemType& e); // 获取栈顶元素
    Status pop(SElemType& e); // 删除栈顶元素
    Status push(SElemType e); // 将元素压入栈中
    Status clear(); // 清理整个栈
    bool is_stack_empty(); // 判断栈是否为空
};
```


该结构体 sqstack 用于实现一个顺序栈，包含指向栈顶和栈底的指针以及栈的大小。它提供了基本的栈操作方法，如入栈、出栈、获取栈顶元素和清空栈。构造函数和析构函数负责初始化和释放栈的内存，确保栈的正确管理。

4.2.4 功能实现

```
Function operation(value_stack, op_stack)
    char ch
    op_stack.pop(ch) // 取出一个运算符

    Switch ch
        Case '|':
            char ch1, ch2
            value_stack.pop(ch2)
            value_stack.pop(ch1)
            value_stack.push((ch1 == 'V' OR ch2 == 'V') ? 'V' : 'F')
            Break

        Case '&':
            char ch1, ch2
            value_stack.pop(ch2)
            value_stack.pop(ch1)
            value_stack.push((ch1 == 'V' AND ch2 == 'V') ? 'V' : 'F')
            Break

        Case '!':
            char get
            value_stack.pop(get)
            value_stack.push(get == 'V' ? 'F' : 'V')
            Break
```

```

    End Switch
End Function

Function main()
    Initialize value_stack, op_stack // 创建值栈和操作符栈
    SElemType ch

    While (ch = getchar()) != EOF
        If ch == '\n' Then
            While NOT op_stack.is_stack_empty()
                operation(value_stack, op_stack) // 处理所有操作符

            SElemType answer
            value_stack.pop(answer) // 获取最终结果
            value_stack.clear() // 清空值栈
            Print "Expression: " + answer
        End If

        Switch ch
            Case 'F', 'V':
                value_stack.push(ch) // 压入布尔值
                Break

            Case '(':

```

```

                Case '(':
                    op_stack.push(ch) // 压入左括号
                    Break

                Case ')':
                    While NOT op_stack.is_stack_empty() AND (op_stack.getTop(get), get != '(')
                        operation(value_stack, op_stack) // 处理操作符
                    op_stack.pop(temp) // 删除左括号
                    Break

                Case '|':
                    While NOT op_stack.is_stack_empty() AND (op_stack.getTop(get), get == '!' OR get == '&')
                        operation(value_stack, op_stack) // 处理优先级更高的操作符
                    op_stack.push(ch) // 压入当前操作符
                    Break

                Case '&':
                    While NOT op_stack.is_stack_empty() AND (op_stack.getTop(get), get == '!')
                        operation(value_stack, op_stack) // 处理优先级更高的操作符
                    op_stack.push(ch) // 压入当前操作符
                    Break

                Case '!':
                    op_stack.push(ch) // 压入非操作符
                    Break
        End Switch
    End While

```

在这个程序中，首先定义了两个栈：一个用于存储布尔值（value_stack），另一个用于存储操作符（op_stack）。当读取到一个字符时，如果是布尔值（'V'或'F'），则将其压入值栈；如果是操作符，则根据其优先级决定是否立即进行运算。遇到右括号时，会弹出操作符栈中的操作符并执行相应的运算，直到遇到左括号为止。对于每个表达式的结束（即遇到换行符），会处理所有剩余的操作符并输出最终结果。整个过程通过循环和条件判断来管理栈

的状态，确保正确计算布尔表达式的值。

4.2.5 心得体会

栈的有效使用：通过使用两个栈分别存储布尔值和操作符，能够有效地管理运算的顺序和优先级。这种设计使得表达式的求值过程更加清晰和高效。

优先级处理：在处理操作符时，合理判断优先级并决定是否立即计算是关键。通过在栈中维护操作符的顺序，可以确保在正确的时机进行运算，从而得到正确的结果。

边界条件的考虑：在处理括号和运算符时，确保正确处理表达式的边界条件（如左括号和右括号的匹配）是非常重要的。这有助于避免潜在的错误，确保程序的健壮性。

调试与测试的重要性：在实现过程中，进行充分的测试和调试是确保代码正确性的关键。通过设计多种测试用例，可以有效验证程序在不同输入下的表现，及时发现并修复问题。

代码可读性与维护性：保持良好的代码结构和注释，使得代码更易于理解和维护。这种良好的编程习惯不仅提高了代码的可读性，也为后续的修改和扩展打下了基础。

5.4 题目四

队列的使用

5.2.1 题目描述

输入一个 $n \times m$ 的 0 1 矩阵，其中 1 表示该位置有东西，0 表示该位置没有东西。所有四邻域联通的 1 算作一个区域，而仅在矩阵边缘联通的不算作区域。该算法常用于细胞计数，目的是求出矩阵中区域的数量，数据范围为 $0 \leq n, m \leq 1000$ 。

5.2.2 题目要求

输入一个 $n \times m$ 的 0 1 矩阵，其中 1 表示该位置有东西，0 表示该位置没有东西。要求计算所有四邻域联通的 1 所组成的区域数量，且仅在矩阵边缘联通的不算作区域。输入格式为第一行包含两个正整数 n 和 m ，接下来的 n 行表示矩阵内容，输出一行表示区域的数量。

5.2.3 数据结构设计

```
#define max_row    1000
#define max_col    1000
using namespace std;

int map[max_row + 10][max_col + 10];
char mark_matrix[max_row + 10][max_col + 10];
```

5.2.4 功能实现

```

Function Markers(row_y, col_x, input_row, input_col)
    // 如果当前位置已经被标记, 返回以避免重复计数
    If mark_matrix[row_y][col_x] == '*' Then
        Return

    // 标记当前位置
    mark_matrix[row_y][col_x] = '*'

    // 向左检查
    If col_x > 0 And map[row_y][col_x - 1] Then
        Call Markers(row_y, col_x - 1, input_row, input_col)

    // 向右检查
    If col_x < input_col - 1 And map[row_y][col_x + 1] Then
        Call Markers(row_y, col_x + 1, input_row, input_col)

    // 向上检查
    If row_y > 0 And map[row_y - 1][col_x] Then
        Call Markers(row_y - 1, col_x, input_row, input_col)

    // 向下检查
    If row_y < input_row - 1 And map[row_y + 1][col_x] Then
        Call Markers(row_y + 1, col_x, input_row, input_col)
End Function

```

```

// 遍历输入矩阵的中间区域
For row from 1 to input_row - 2 Do
    For col from 1 to input_col - 2 Do
        // 如果当前位置是细胞且未被标记
        If map[row][col] And mark_matrix[row][col] != '*' Then
            area = area + 1 // 增加区域计数
            Call Markers(row, col, input_row, input_col) // 标记该区域
        End If
    End For
End For

// 输出区域数
Print area

```

Markers 函数：该函数用于递归地标记与当前细胞相连的所有细胞，避免重复计数。首先检查当前位置是否已被标记，如果已标记则返回；否则，将当前位置标记为已访问。

边界检查：在标记过程中，函数会检查当前位置的四个邻居（左、右、上、下），如果邻居是细胞（值为 1）且未超出边界，则递归调用 Markers 函数继续标记。

区域计数：在主程序中，遍历输入矩阵的中间区域（避免边界），每当发现一个未被标记的细胞时，区域计数增加，并调用 Markers 函数标记该区域的所有细胞，最后输出区域的总数。

5.2.5 调试分析（遇到的问题 and 解决方法）

在解决这个区域计数问题时，我遇到了以下几个困难及其解决方法：

递归深度限制：在处理较大的矩阵时，递归调用可能导致栈溢出。为了解决这个问题，我考虑使用迭代方法（如使用栈或队列）来替代递归，从而避免深度限制。

边界条件处理：在遍历矩阵时，确保不访问边界外的元素是一个挑战。通过在循环中设置适当的边界条件（如从 1 到 `input_row - 2`）来限制访问范围，有效避免了越界错误。

标记机制的设计：如何有效地标记已访问的细胞以防止重复计数是一个关键问题。我使用了一个额外的标记矩阵 `mark_matrix`，在访问细胞时将其标记为已访问，从而确保每个区域只被计数一次。

邻域检查的实现：在检查细胞的四个邻域时，确保逻辑正确且不遗漏任何连接的细胞是一个挑战。通过逐一检查每个方向并使用条件语句进行判断，确保了所有相连的细胞都能被正确标记。

调试与测试：在实现过程中，调试和验证代码的正确性是必要的，尤其是在处理复杂的输入数据时。通过编写测试用例和使用随机生成的数据进行验证，确保了算法的正确性和鲁棒性。

5.2.6 心得体会

在解决区域计数问题的过程中，我深刻体会到使用递归和栈结构的重要性，这使得处理相连区域变得更加高效和直观。通过合理设计标记机制，能够有效避免重复计数，从而提高算法的准确性。最后，充分的测试和调试是确保代码质量的关键，能够帮助我及时发现并修复潜在的问题。

6.2 题目五

队列的最大值

6.2.1 题目描述

题目要求设计一个数据结构来实现一个队列，支持入队、出队和获取最大值的操作。具体来说，队列需要提供三个基本操作：`Enqueue(v)`将元素 `v` 入队，`Dequeue()`删除并返回队首元素，以及 `GetMax()`返回队列中的最大元素。为了提高效率，`GetMax()`操作的时间复杂度需要尽可能低，以确保在规定的时间内完成所有操作。

6.2.2 题目要求

题目要求设计一个队列数据结构，支持三种基本操作：入队 (`Enqueue(v)`)、出队 (`Dequeue()`) 和获取最大值 (`GetMax()`)。在实现中，需要确保 `GetMax()`操作的时间复杂度尽可能低，并且在规定的时间内完成所有操作。输入包括队列的容量和一系列操作指令，要求在队列满或空时输出相应的提示信息，并在结束时输出队列中的所有元素。

6.2.3 数据结构设计


```

// 定义元素类型
typedef int ElemType;
// 定义状态类型
typedef int Status;
// 定义状态常量
#define S_OVERFLOW -3 // 内存溢出
#define FULL -2 // 栈满
#define EMPTY -1 // 栈空
#define OK 0 // 操作成功
// 顺序栈结构体
struct SqStack
{
private:
    ElemType* base; // 栈底指针, 指向栈的起始位置
    ElemType* top; // 栈顶指针, 指向栈的当前顶端
    int stacksize; // 栈的大小
public:
    SqStack(); // 构造空栈
    ~SqStack(); // 销毁已有的栈
    Status ClearStack(); // 清空栈
    Status Top(ElemType& e); // 获取栈顶元素
    Status Pop(ElemType& e); // 弹出栈顶元素
    Status Push(ElemType e); // 入栈
    bool StackEmpty(); // 判断栈是否为空
};

```

```

// 带最大值的顺序栈结构体
class SqStack_WithMax
{
private:
    SqStack Value; // 存储实际值的栈
    SqStack Max; // 存储当前最大值的栈
public:
    Status Top(ElemType& e); // 获取栈顶元素
    Status Pop(ElemType& e); // 弹出栈顶元素
    Status Push(ElemType e); // 入栈
    Status GetMax(ElemType& e); // 获取栈中最大值
    bool StackEmpty(); // 判断栈是否为空
};

```

```

// 带最大值的队列结构体
class SqQueue_WithMax
{
private:
    SqStack_WithMax a;           // 队列的入队栈
    SqStack_WithMax b;           // 队列的出队栈
    int ElemNum = 0;             // 当前队列中的元素数量

public:
    Status EnQueue(ElemType e);  // 在队尾插入元素
    Status DeQueue(ElemType& e); // 弹出队首元素
    Status GetMax(ElemType& e);  // 获取队列最大值
    bool QueueEmpty();           // 判断队列是否为空
};

```

SqStack 结构体：实现了一个顺序栈，包含基本的栈操作，如入栈、出栈、获取栈顶元素和判断栈是否为空。使用 base 和 top 指针来管理栈的存储和访问。

SqStack_WithMax 类：在 SqStack 的基础上扩展，增加了一个辅助栈 Max 来存储当前栈中的最大值，从而在 O(1)时间内获取最大值。

SqQueue_WithMax 类：实现了一个队列，使用两个 SqStack_WithMax 实例来管理入队和出队操作，同时支持获取队列中的最大值。通过维护元素数量 ElemNum 来判断队列是否为空。

6.2.4 功能实现

```

// 公共方法
Function SqStack() // 构造空栈
    base = new ElemType[MAX_SIZE]
    If base is null Then
        exit(SOVERFLOW)
    top = base
    stacksize = MAX_SIZE

Function ~SqStack() // 析构函数
    If base is not null Then
        delete base
    stacksize = 0

Function ClearStack() // 清空栈
    If base is not null Then
        delete base
    base = new ElemType[MAX_SIZE]
    If base is null Then
        exit(SOVERFLOW)
    top = base
    stacksize = MAX_SIZE
    return OK

```

```

Function Top(ElemType& e) // 获取栈顶元素
    If top == base Then
        return EMPTY
    e = *(top - 1)
    return OK

Function Pop(ElemType& e) // 弹出栈顶元素
    If top == base Then
        return EMPTY
    e = *(--top)
    return OK

Function Push(ElemType e) // 入栈
    If top - base >= stacksize Then
        return FULL
    *top = e
    top++
    return OK

Function StackEmpty() // 判断栈是否为空
    return base == top

```

```
// 带最大值的顺序栈结构体
class SqStack_WithMax
// 私有成员
    SqStack Value      // 存储实际值的栈
    SqStack Max        // 存储当前最大值的栈

// 公共方法
    Function Top(ElemType& e) // 获取栈顶元素
        return Value.Top(e)

    Function Pop(ElemType& e) // 弹出栈顶元素
        ElemType max
        Max.Top(max)
        If Value.StackEmpty() Then
            return EMPTY
        Value.Pop(e)
        If e == max Then
            Max.Pop(max)
        return OK

    Function Push(ElemType e) // 入栈
        ElemType max
        Max.Top(max)
        If Value.Push(e) == FULL Then
```

```
Function Push(ElemType e) // 入栈
    ElemType max
    Max.Top(max)
    If Value.Push(e) == FULL Then
        return FULL
    If Max.StackEmpty() or e >= max Then
        Max.Push(e)
    return OK

Function GetMax(ElemType& e) // 获取栈中最大值
    return Max.Top(e)

Function StackEmpty() // 判断栈是否为空
    return Value.StackEmpty()
```

```
Function GetMax(ElemType& e) // 获取队列最大值
    If ElemNum == 0 Then
        return EMPTY
    e = 0x80000000 // 先置为最小值
    ElemType e2
    If not b.StackEmpty() Then
        b.GetMax(e)
    If not a.StackEmpty() Then
        a.GetMax(e2)
    e = max(e, e2)
    return OK

Function QueueEmpty() // 判断队列是否为空
    return ElemNum == 0
```

6.2.5 调试分析（遇到的问题 and 解决方法）

栈的溢出问题：在进行入队操作时，如果队列已满，可能会导致栈溢出。为了解决这个问题，我在 Push 方法中添加了对栈满的检查，并返回相应的状态，以避免继续执行入队操作。

出队时的元素转移：在出队操作中，当出队栈为空时，需要将入队栈的元素转移到出队栈中。这一过程容易出错，特别是在处理元素顺序时。为了解决这个问题，我确保在转移元素时使用循环将所有元素从入队栈弹出并推入出队栈，确保顺序正确。

获取最大值的效率：在获取队列最大值时，初始设计可能需要遍历整个队列，导致时间复杂度较高。为了解决这个问题，我引入了一个辅助栈来存储当前最大值，这样可以在 $O(1)$ 时间内获取最大值，显著提高了效率。

边界条件的处理：在实现过程中，处理队列为空或满的情况是非常重要的。通过在每个

操作中添加条件判断，确保在队列为空时返回相应的提示信息，避免了潜在的错误。

调试与测试：在实现完功能后，进行充分的测试是确保代码正确性的关键。我编写了多种测试用例，涵盖了不同的输入情况，以验证程序的稳定性和正确性，及时发现并修复了潜在的问题。

6.2.6 心得体会

数据结构的选择：通过使用两个栈来实现队列的功能，不仅能够有效地管理元素的入队和出队操作，还能在 $O(1)$ 时间内获取当前队列的最大值。这种设计充分利用了栈的后进先出特性，使得队列的实现更加高效。

复杂度优化：在处理最大值的获取时，使用辅助栈来存储当前最大值的逻辑非常关键。这种方法避免了每次获取最大值时都需要遍历整个队列，从而显著降低了时间复杂度，提高了程序的性能。

边界条件的处理：在实现过程中，考虑到队列为空或满的情况是非常重要的。通过合理的条件判断和状态返回，可以确保程序在各种情况下都能稳定运行，避免了潜在的错误和异常。

调试与测试的重要性：在实现完功能后，进行充分的测试是确保代码正确性的关键。通过设计多种测试用例，可以有效地验证程序在不同输入下的表现，及时发现并修复问题。

代码的可读性与维护性：在编写代码时，保持良好的注释和清晰的结构有助于后续的维护和理解。良好的代码风格不仅提高了可读性，也使得团队协作更加顺畅。

3.实验总结

在本次实验中，我们实现了一个支持最大值查询的队列数据结构，主要通过两个栈来管理队列的入队和出队操作。以下是实验的主要收获和总结：

数据结构的设计：通过使用两个栈（一个用于入队，另一个用于出队），我们有效地模拟了队列的行为。这种设计不仅简化了队列的实现，还使得获取最大值的操作变得高效。

时间复杂度优化：引入辅助栈来存储当前最大值，使得获取最大值的操作能够在 $O(1)$ 时间内完成，显著提高了程序的性能。这一优化在处理大量数据时尤为重要，确保了程序在规定时间内完成所有操作。

边界条件的处理：在实现过程中，充分考虑了队列为空和满的情况，确保在这些情况下能够返回正确的提示信息。这种细致的边界条件处理提高了程序的健壮性，避免了潜在的运行时错误。

调试与测试的重要性：通过编写多种测试用例，验证了程序在不同输入下的表现，及时发现并修复了潜在的问题。这一过程强调了测试在软件开发中的重要性，确保了代码的正确性和稳定性。

代码可读性与维护性：在编写代码时，保持良好的注释和清晰的结构，使得代码更易于理解和维护。这种良好的编程习惯不仅提高了代码的可读性，也为后续的修改和扩展打下了基础。